

Assignment 7

Josh Suber
RIN: 662038070

CSCI 4100

Contents

1	Linear Regression for classification followed by pocket for improvement	3
1.1	Plots	3
1.2	E_{in} and E_{test}	4
1.3	3rd order polynomial transform	5
1.4	E_{in} and E_{test} (3rd order polynomial transform)	6
2	Logistic regression for classification using gradient descent	7
2.1	Plots	7
2.2	E_{in} and E_{test}	8
2.3	3rd order polynomial transform	9
2.4	E_{in} and E_{test} (3rd order polynomial transform)	10
3	Logistic regression for classification using stochastic gradient descent	11
3.1	Plots	11
3.2	E_{in} and E_{test}	12
3.3	3rd order polynomial transform	13
3.4	E_{in} and E_{test} (3rd order polynomial transform)	14
4	Error Bound Calculation	15
5	Recommendation	16
6	Question 2	17
6.1	Part (a): Gradient Descent with Different Learning Rates . . .	17
6.2	Part (b): Results from Different Starting Points	18
6.3	Conclusion	19
7	Problem 3.16	19
7.1	(a)	19
7.2	(b)	20
7.3	(c)	20
7.4	Interpretation	21

1 Linear Regression for classification followed by pocket for improvement

1.1 Plots

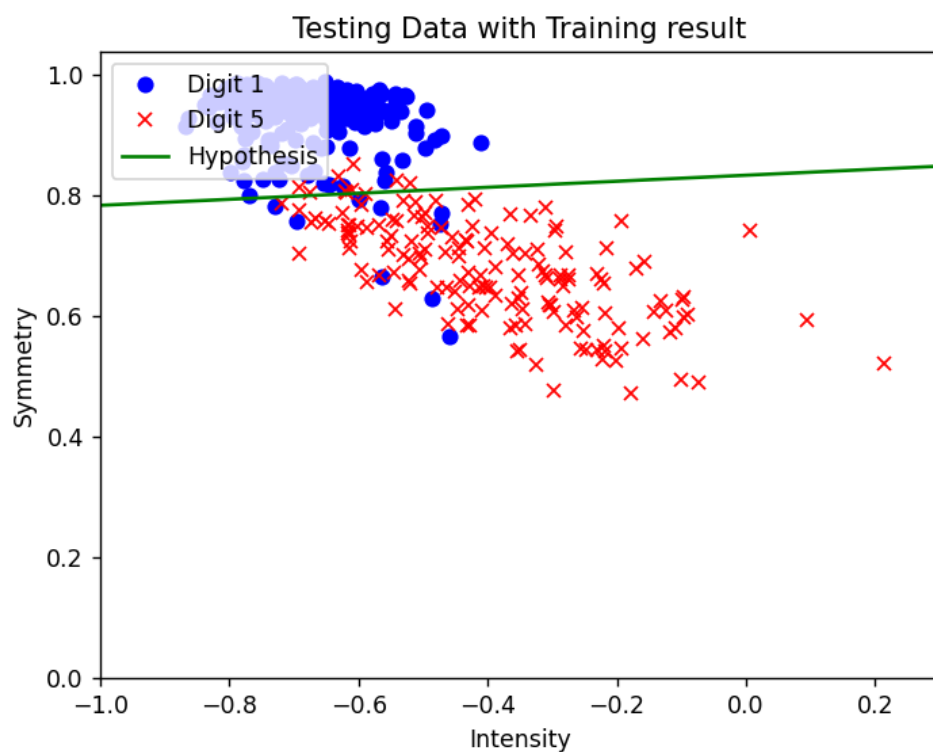


Figure 1: Plot of Separator(test)

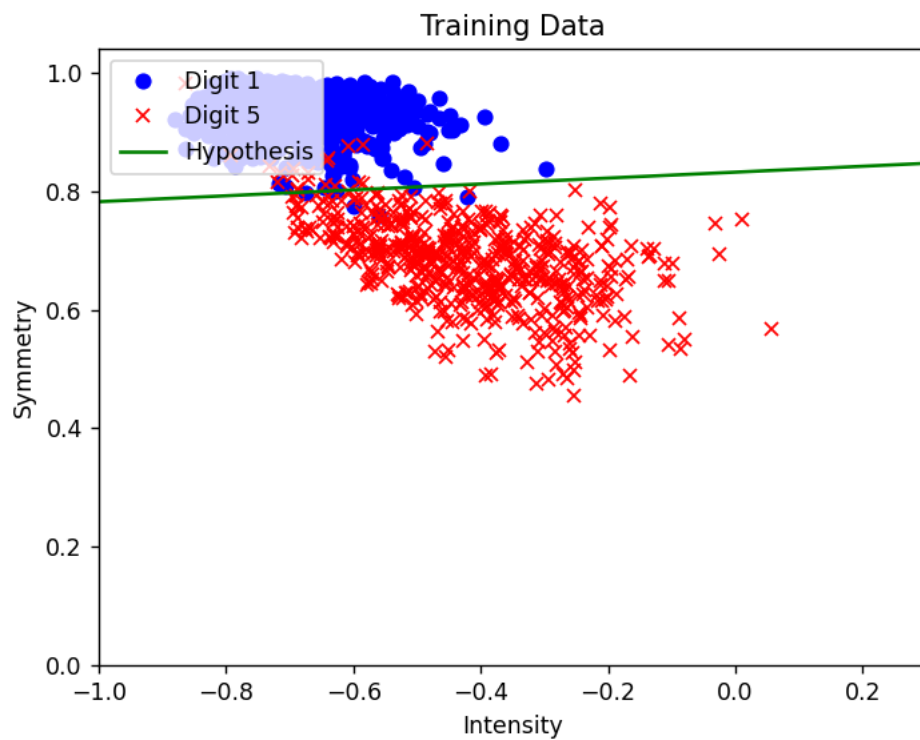


Figure 2: Plot of Separator(train)

1.2 E_{in} and E_{test}

$$E_{in} = 0.016656$$

$$E_{test} = 0.042453$$

1.3 3rd order polynomial transform

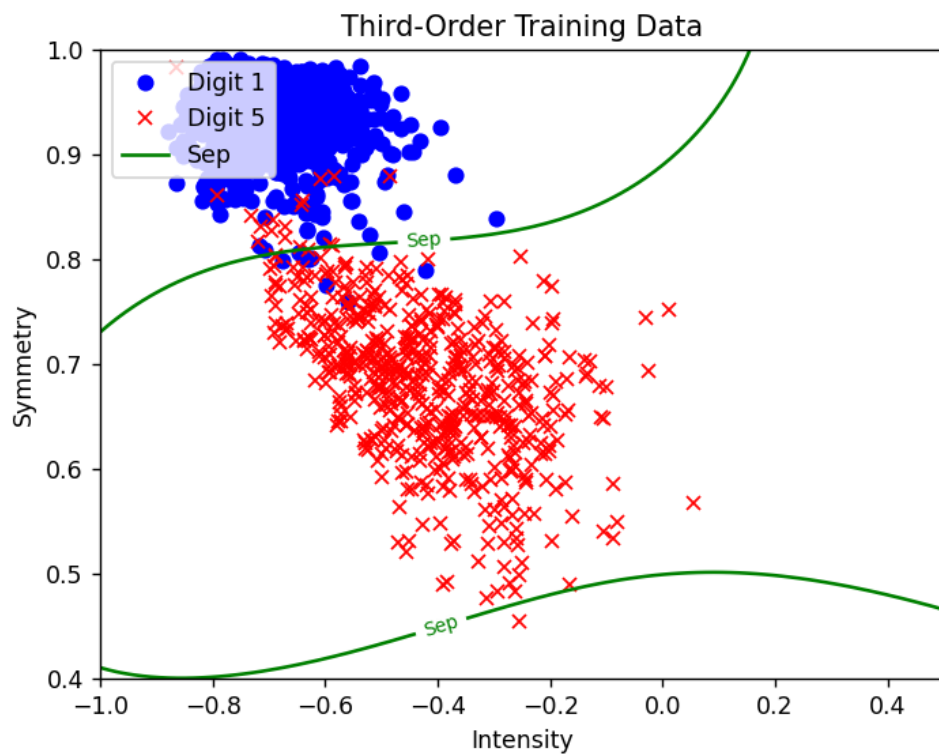


Figure 3: Plot of Separator(test) for 3rd Polynomial

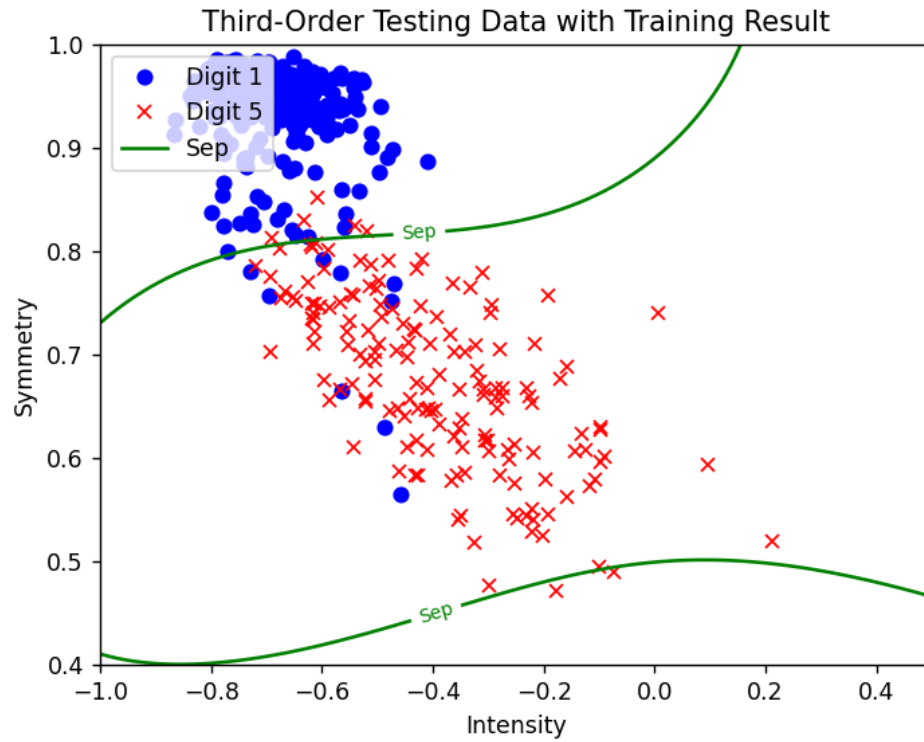


Figure 4: Plot of Separator(train) for 3rd Polynomial

1.4 E_{in} and E_{test} (3rd order polynomial transform)

$$E_{in} = 0.0172966$$

$$E_{test} = 0.037736$$

2 Logistic regression for classification using gradient descent

2.1 Plots

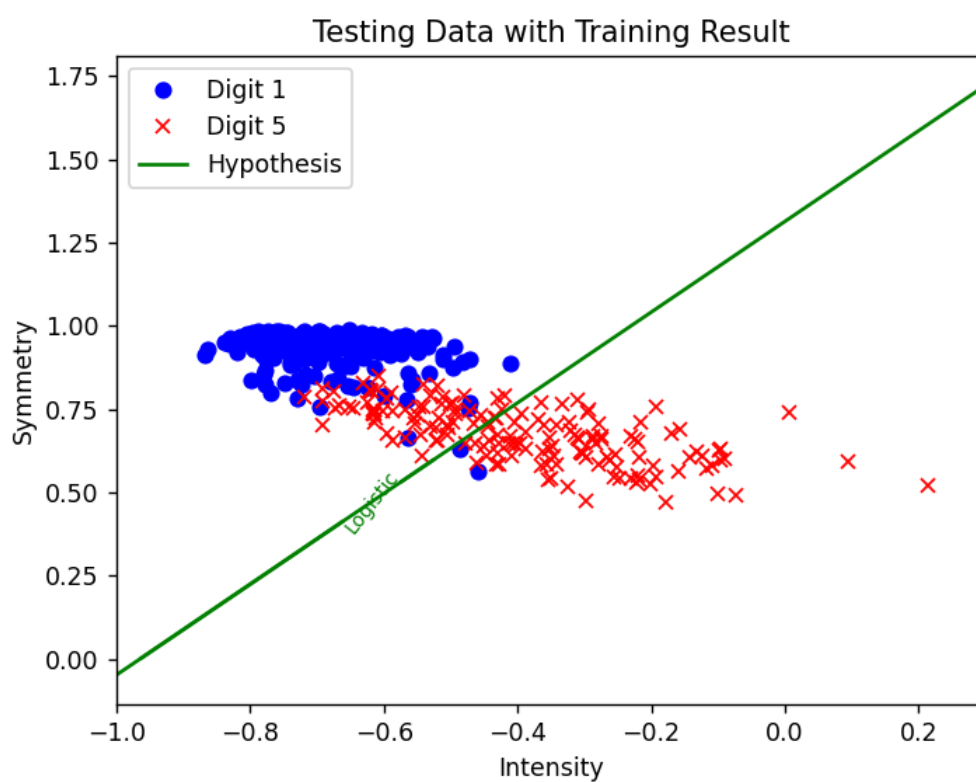


Figure 5: Plot of Separator(test)

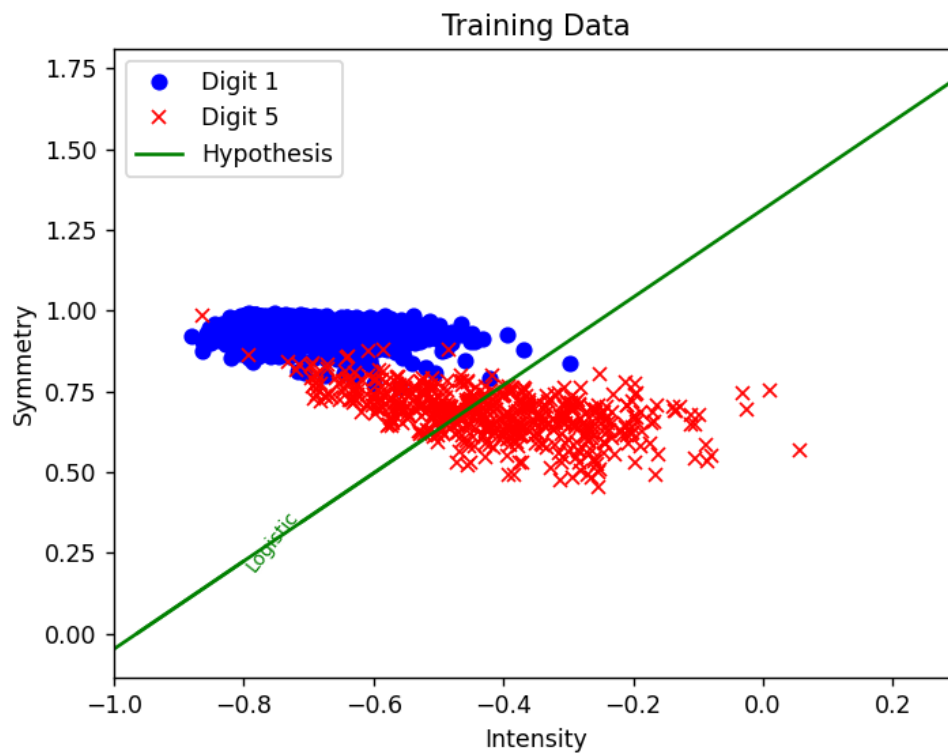


Figure 6: Plot of Separator(train)

2.2 E_{in} and E_{test}

$$E_{in} = 0.15183$$

$$E_{test} = 0.15566$$

2.3 3rd order polynomial transform



Figure 7: Plot of Separator(test) for 3rd Polynomial

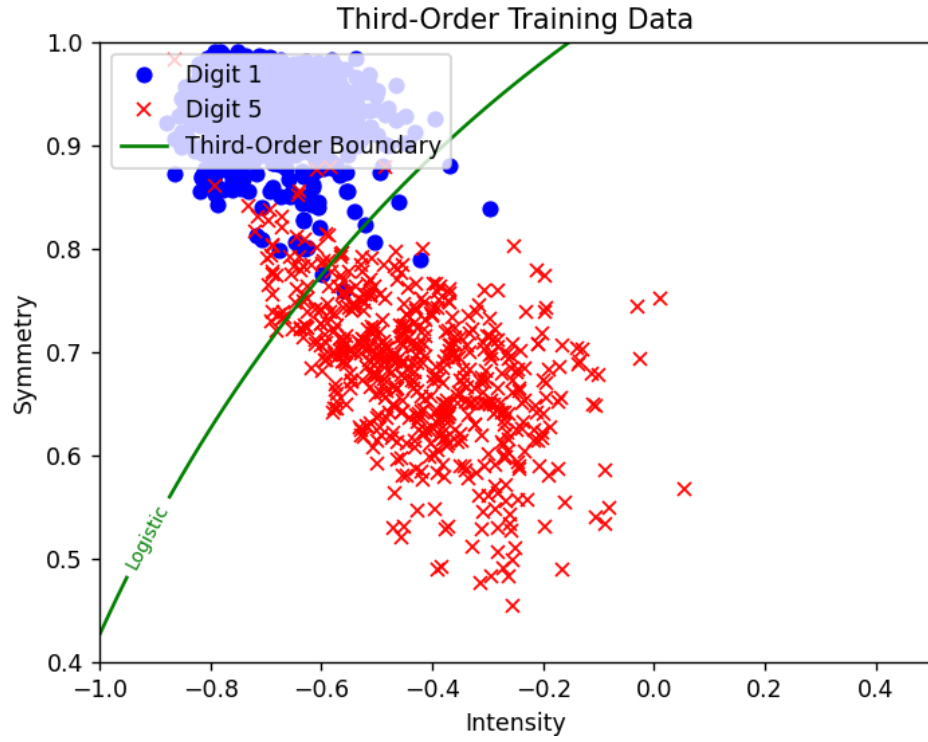


Figure 8: Plot of Separator(train) for 3rd Polynomial

2.4 E_{in} and E_{test} (3rd order polynomial transform)

$$E_{in} = 0.035234$$

$$E_{test} = 0.0542453$$

3 Logistic regression for classification using stochastic gradient descent

3.1 Plots

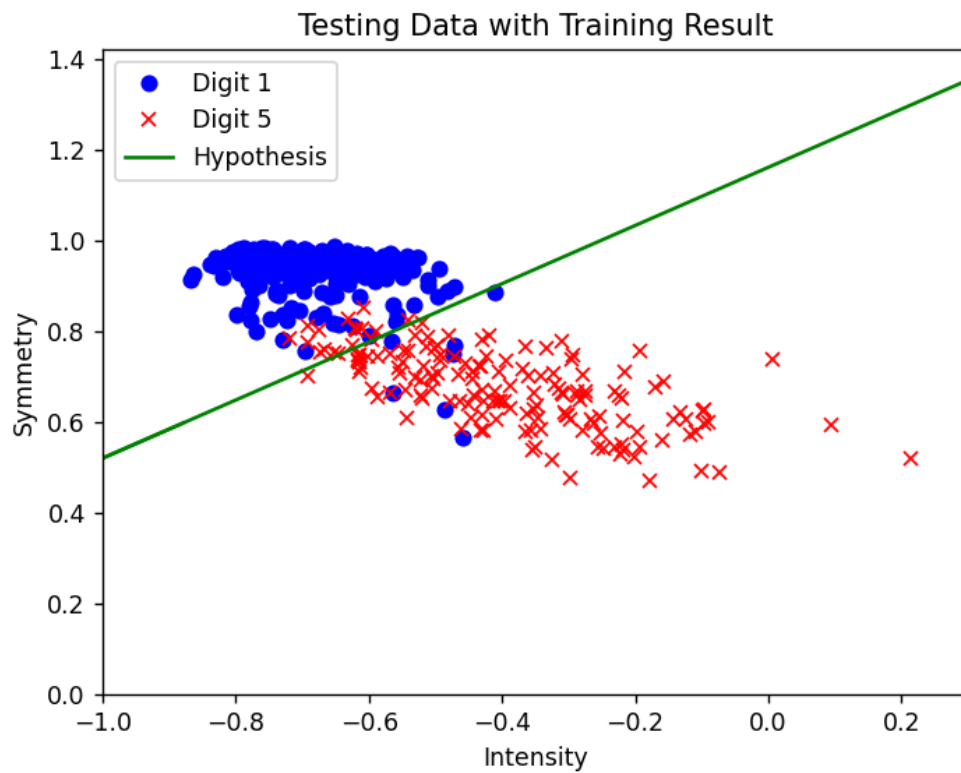


Figure 9: Plot of Separator(test)

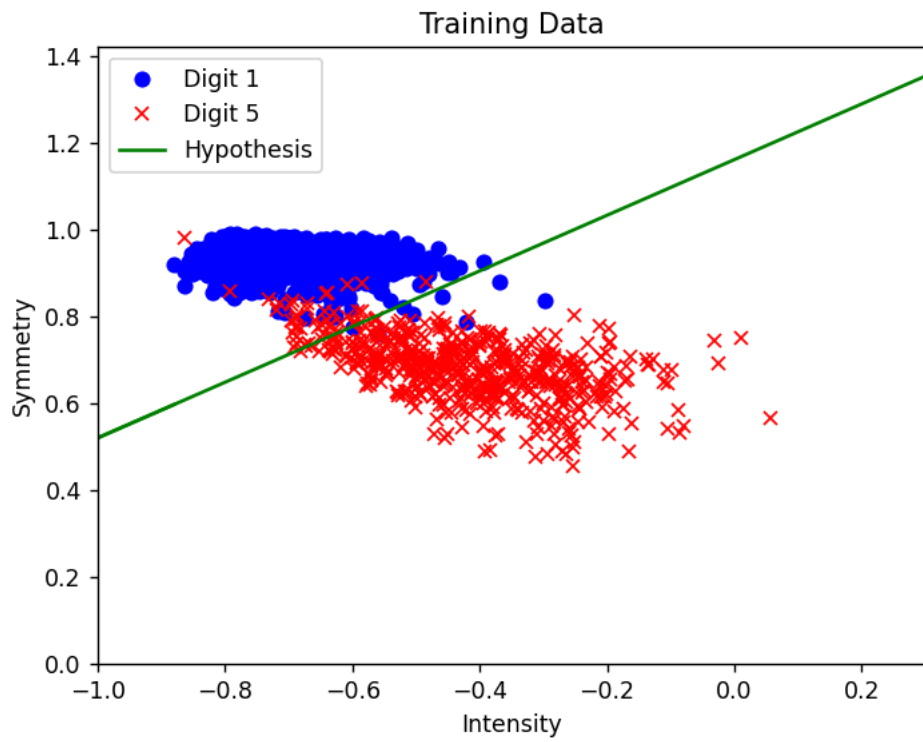


Figure 10: Plot of Separator(train)

3.2 E_{in} and E_{test}

$$E_{in} = 0.0345932$$

$$E_{test} = 0.05660377$$

3.3 3rd order polynomial transform



Figure 11: Plot of Separator(test) for 3rd Polynomial

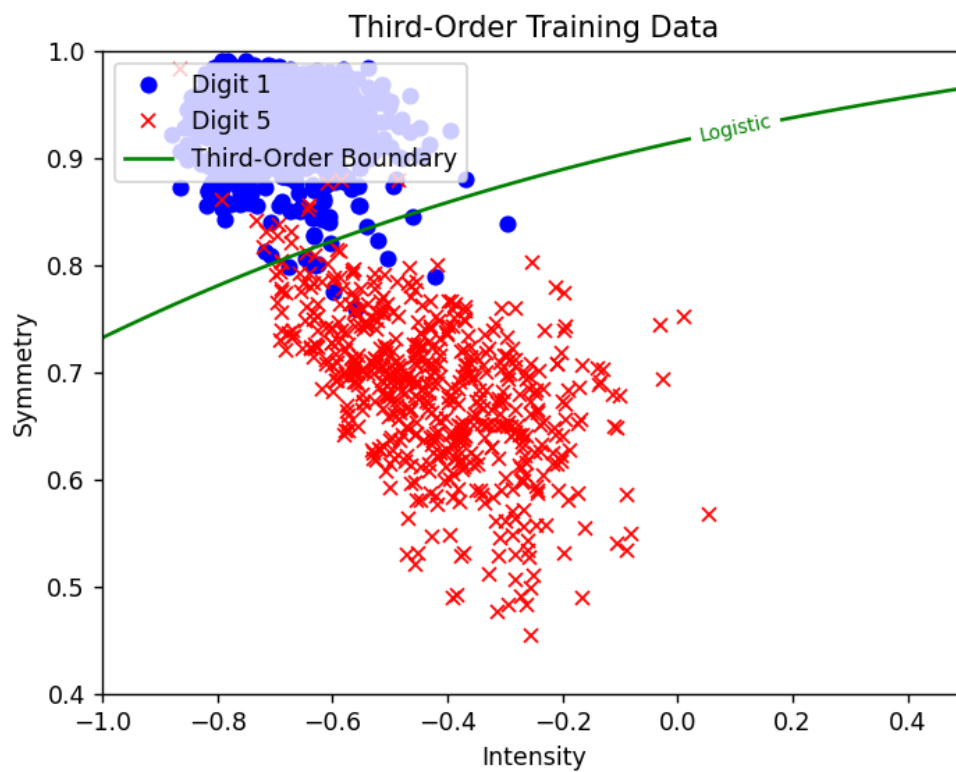


Figure 12: Plot of Separator(train) for 3rd Polynomial

3.4 E_{in} and E_{test} (3rd order polynomial transform)

$$E_{in} = 0.0172966$$

$$E_{test} = 0.0330189$$

4 Error Bound Calculation

1. Best Method for Non-3rd Order Polynomial: Linear Regression with Pocket

Using E_{in} (with $N = 1561$):

$$\begin{aligned}E_{in} &= 0.016656, \\ \delta &= 0.05, \\ H &= 2 \text{ (assumed)}.\end{aligned}$$

The bound is calculated using:

$$E_{out} \leq E_{in} + \sqrt{\frac{8}{1561} \ln \left(\frac{4 \times (2 \times 2)^3 + 1}{0.05} \right)}.$$

Calculating this yields:

$$E_{out} \leq 0.2259.$$

Using E_{test} (with $N = 424$):

$$\begin{aligned}E_{test} &= 0.042453, \\ \delta &= 0.05, \\ H &= 2 \text{ (assumed)}.\end{aligned}$$

The bound is calculated using:

$$E_{out} \leq E_{test} + \sqrt{\frac{1}{2 \times 424} \ln \left(\frac{1 \times 2}{0.05} \right)}.$$

Calculating this yields:

$$E_{out} \leq 0.0849.$$

Therefore, the bound on E_{in} is better.

2. Best Method for 3rd Order Polynomial: Logistic Regression with Stochastic Gradient Descent

Using E_{in} (with $N = 1561$):

$$\begin{aligned}E_{in} &= 0.0172966, \\ \delta &= 0.05, \\ H &= 2 \text{ (assumed)}.\end{aligned}$$

The bound is calculated using:

$$E_{out} \leq E_{in} + \sqrt{\frac{8}{1561} \ln \left(\frac{4 \times (2 \times 2)^3 + 1}{0.05} \right)}.$$

Calculating this yields:

$$E_{out} \leq 0.2259.$$

Using E_{test} (with $N = 424$):

$$E_{test} = 0.0330189,$$

$$\delta = 0.05,$$

$$H = 2 \text{ (assumed).}$$

The bound is calculated using:

$$E_{out} \leq E_{test} + \sqrt{\frac{1}{2 \times 424} \ln \left(\frac{1 \times 2}{0.05} \right)}.$$

Calculating this yields:

$$E_{out} \leq 0.0849.$$

Therefore, the bound on E_{test} is better.

5 Recommendation

Based on the data and analysis, I would recommend using the linear model with the 3rd order polynomial transformation. Although both models perform well on the in-sample error (E_{in}), the 3rd order polynomial model achieves a notably lower test error ($E_{test} = 0.0330189$ compared to $E_{test} = 0.042453$ for the model without the polynomial transformation). This lower test error suggests that the 3rd order polynomial model generalizes better to unseen data, which is crucial for any model intended for real-world use. The bounds on the out-of-sample error E_{out} further support this choice; both models have similar bounds when calculated with E_{in} and E_{test} , but the 3rd order polynomial model slightly edges out in reliability. While the polynomial transformation adds complexity, in this case, it enhances the model's performance without overfitting, as evidenced by its low test error. Given the lower test error and solid out-of-sample performance bounds, I believe the 3rd order polynomial model is the better choice for deployment, ensuring better generalization to new data.

6 Question 2

The goal is to minimize the function:

$$f(x, y) = x^2 + 2y^2 + 2 \sin(2\pi x) \sin(2\pi y)$$

using gradient descent with two different learning rates ($\eta = 0.01$ and $\eta = 0.1$) over 50 iterations.

6.1 Part (a): Gradient Descent with Different Learning Rates

Learning Rate $\eta = 0.01$: The plot below shows the behavior of the cost function over 50 iterations with a learning rate of $\eta = 0.01$. The cost function decreases smoothly and stabilizes within the first 10 iterations, converging to a minimum value around -1.82 . This indicates that a smaller learning rate allows for stable convergence, with the algorithm gradually approaching the minimum without oscillations.

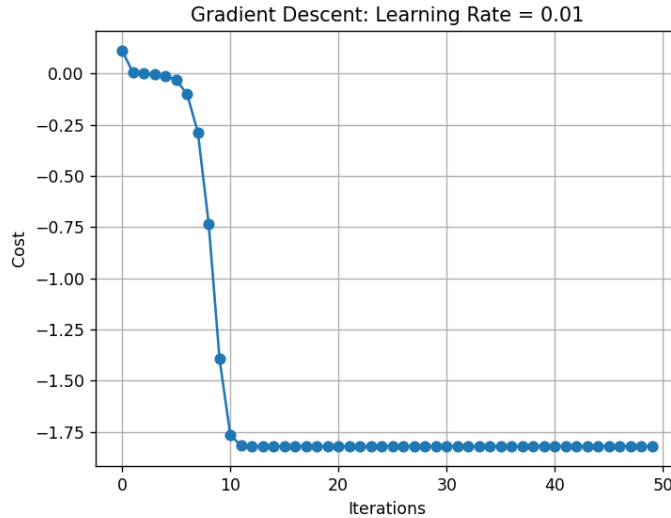


Figure 13: Gradient Descent: Learning Rate $\eta = 0.01$

Learning Rate $\eta = 0.1$: The plot below shows the cost function over 50 iterations with a learning rate of $\eta = 0.1$. The cost function exhibits

significant oscillations and does not converge smoothly. The high learning rate causes the algorithm to overshoot, preventing stable convergence to the minimum. This demonstrates that a high learning rate can lead to instability in gradient descent.

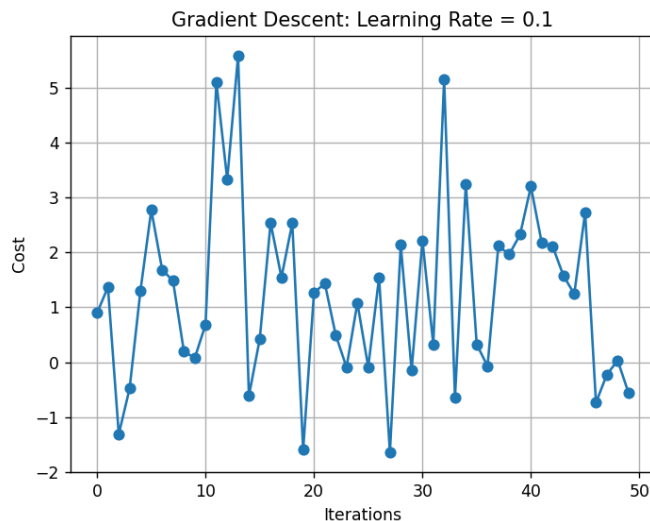


Figure 14: Gradient Descent: Learning Rate $\eta = 0.1$

6.2 Part (b): Results from Different Starting Points

To explore the impact of different starting points on finding the minimum, gradient descent was initialized from several points. The table below shows the final x , y coordinates and minimum costs achieved for each starting point, using both learning rates.

The results show that:

- Different starting points yield different minimum values and locations, which is expected given the non-convex nature of $f(x, y)$.
- The algorithm converged to various local minima depending on the starting point, with some results near -1.82 (the lowest achieved value) and others around -1.33 or -1.69 .

Starting Point	Learning Rate	Final x	Final y	Minimum Cost
(0.1, 0.1)	0.10	0.2446	-0.3031	-1.6452
(0.1, 0.1)	0.01	0.2438	-0.2379	-1.8201
(1, 1)	0.10	0.1973	-0.2668	-1.6997
(1, 1)	0.01	1.2181	0.7128	0.5933
(-0.5, -0.5)	0.10	0.2863	-0.3315	-1.3965
(-0.5, -0.5)	0.01	-0.7314	-0.2379	-1.3325
(-1, -1)	0.10	-0.1973	0.2668	-1.6997
(-1, -1)	0.01	-1.2181	-0.7128	0.5933

Figure 15: Final Results from Different Starting Points

- This variability highlights the challenge in optimization: finding the global minimum of a non-convex function is difficult because gradient descent can get "trapped" in local minima.

6.3 Conclusion

The observations from part (a) show that a smaller learning rate provides stability, while a larger learning rate causes oscillations and instability. Part (b) illustrates that the initial starting point in gradient descent significantly impacts the convergence result, underscoring why finding the global minimum in non-convex optimization problems is challenging.

7 Problem 3.16

7.1 (a)

To define the expected cost of accepting or rejecting a person, we calculate the costs based on the given probability $g(x)$, which is the probability of $y = +1$ given x .

1. Cost of Accepting ($\text{cost}(\text{accept})$): When we choose to accept, the cost depends on the probability of mistakenly accepting an intruder. The cost is:

$$\text{cost}(\text{accept}) = P[\text{correct}] \cdot 0 + P[\text{intruder}] \cdot C_a = (1 - g(x)) \cdot C_a$$

2. Cost of Rejecting (cost(reject)): When we decide to reject, the cost is influenced by the chance of incorrectly rejecting a correct person. This gives:

$$\text{cost}(\text{reject}) = P[\text{correct}] \cdot C_r + P[\text{intruder}] \cdot 0 = g(x) \cdot C_r$$

Thus, we have shown that:

$$\text{cost}(\text{accept}) = (1 - g(x)) \cdot C_a, \quad \text{cost}(\text{reject}) = g(x) \cdot C_r$$

7.2 (b)

To find the threshold for deciding to accept or reject, we set up a comparison where the cost of accepting must be less than or equal to the cost of rejecting:

$$(1 - g(x)) \cdot C_a \leq g(x) \cdot C_r$$

Rearranging the inequality:

$$C_a - g(x) \cdot C_a \leq g(x) \cdot C_r$$

$$C_a \leq g(x) \cdot (C_r + C_a)$$

$$g(x) \geq \frac{C_a}{C_a + C_r}$$

Thus, we establish the threshold κ as:

$$\kappa = \frac{C_a}{C_a + C_r}$$

7.3 (c)

Using the cost matrices provided in Example 1.1, we can compute κ for the Supermarket and CIA scenarios.

- Supermarket Case: Given $C_a = 1$ and $C_r = 10$:

$$\kappa = \frac{1}{10 + 1} = 0.0909$$

- CIA Case: Given $C_a = 1000$ and $C_r = 1$:

$$\kappa = \frac{1000}{1000 + 1} = 0.9990$$

7.4 Interpretation

In the Supermarket example, the relatively low cost of false rejection (C_r) and higher willingness to accept people leads to a lower threshold ($\kappa = 0.0909$), meaning the supermarket will accept individuals even with a modest confidence level that they are correct.

In contrast, the CIA example has a high cost of false acceptance (C_a), leading to a high threshold ($\kappa = 0.9990$), indicating that the CIA requires near-certainty that the person is genuine before accepting them.